

Iterative metoder 2

Jonas J. Harang
30. oktober 2024



NTNU

Tema og læringsutbytte

Sauer 2.6: Metoder for positiv-definitte system

- Symmetrisk positiv definitte matriser
- Cholesky-faktorisering
- Konjugerte gradienters metode

Du skal kunne:

- Bestemme om et matrise er symmetrisk og/eller positiv definitt eller ikke
- Utføre Cholesky-faktorisering av mindre matriser
- Forklare i grove trekk hvordan konjugerte gradienters metode fungerer
- Redegjøre for kondisjonstallets betydning i CG ..
- .. og hva det har med forkondisjonering å gjøre



NTNU

Symmetriske matriser

Definisjon

En matrise A kalles symmetrisk dersom:

$$A^T = A$$

Eksempel

- Matrisen $A = \begin{bmatrix} 4 & -2 \\ -2 & 5 \end{bmatrix}$ er symmetrisk
- Matrisen $A = \begin{bmatrix} 3 & -2 \\ -1 & 3 \end{bmatrix}$ er ikke symmetrisk
- Matrisen $A = \begin{bmatrix} 4 & -2 & 0 \\ -2 & 5 & 0 \\ 0 & 0 & 0 \end{bmatrix}$ er ikke symmetrisk

En symmetrisk matrise er alltid kvadratisk



NTNU

Positiv definite symmetriske matriser

Definisjon

En en symmetrisk $n \times n$ matrise A kalles positivt definit dersom:

$$v^T A v > 0$$

for alle vektorer v

Eksempel 1

Matrisen $A = \begin{bmatrix} 2 & -2 \\ -2 & 3 \end{bmatrix}$ er symmetrisk positiv definit

Eksempel 2

Matrisen $A = \begin{bmatrix} 2 & 3 \\ 3 & 4 \end{bmatrix}$ er symmetrisk, men ikke positiv definit

Forklaring

Eksempel 1

Matrisen $A = \begin{bmatrix} 2 & -2 \\ -2 & 3 \end{bmatrix}$ er symmetrisk
positiv definit

Forklaring

Eksempel 2

Matrisen $A = \begin{bmatrix} 2 & 3 \\ 3 & 4 \end{bmatrix}$ er symmetrisk, men ikke positiv definit



NTNU

Egenskaper til positivt definitte matriser I

Teorem 1

*En symmetrisk $n \times n$ matrise er positiv definit
hvis og bare hvis alle egenverdier er positive*

Eksempel

$$A = \begin{bmatrix} 3 & 2 & 1 \\ 2 & 6 & 2 \\ 1 & 2 & 3 \end{bmatrix}$$

$$\det(A - \lambda I) = -\lambda^3 + 12\lambda^2 - 36\lambda + 32$$



NTNU

Egenskaper til positivt definitte matriser II

Eksempel

Definisjon

En prinsipal undermatrise av en $n \times n$ matrise A er en kvadratisk matrise der alle diagonalelementer er diagonalelementer til A

$$A = \begin{bmatrix} 3 & 2 & 1 \\ 2 & 6 & 2 \\ 1 & 2 & 3 \end{bmatrix}$$

Teorem 2

Alle prinsipale undermatriser til en symmetrisk positivt definit matrise A er positivt definit



NTNU

Egenskaper til positivt definitte matriser III

Teorem 3

Hvis A er en positivt definitt symmetrisk $n \times n$ matrise og X er en $n \times m$ matrise med full rank og $m < n$ så er

$$X^T A X$$

også positivt definitt

Eksempel

$$A = \begin{bmatrix} 3 & 2 & 1 \\ 2 & 6 & 2 \\ 1 & 2 & 3 \end{bmatrix}$$

$$X = \begin{bmatrix} 1 & 0 \\ 0 & 1 \\ 0 & 0 \end{bmatrix}$$



NTNU

Egenskaper til positivt definitte matriser IV

Teorem 4

Hvis A er en positivt definitt symmetrisk $n \times n$ matrise så er alle diagonalelementer i $A > 0$

Eksempel

$$A = \begin{bmatrix} 3 & 2 & 1 \\ 2 & -6 & 2 \\ 1 & 2 & 3 \end{bmatrix}$$

$$x = \begin{bmatrix} 0 \\ 1 \\ 0 \end{bmatrix}$$



NTNU

Cholesky-faktorisering

Vi ser på en øvre triangulær matrise U :

$$U = \begin{bmatrix} a & b & c \\ 0 & d & e \\ 0 & 0 & f \end{bmatrix}$$

og på produktet:

$$\begin{aligned} U^T U &= \begin{bmatrix} a & 0 & 0 \\ b & d & 0 \\ c & e & f \end{bmatrix} \cdot \begin{bmatrix} a & b & c \\ 0 & d & e \\ 0 & 0 & f \end{bmatrix} \\ &= \begin{bmatrix} a^2 & ab & ac \\ ab & b^2 + d^2 & bc + de \\ ac & bc + de & f^2 + e^2 + c^2 \end{bmatrix} \end{aligned}$$



NTNU

Cholesky-faktorisering

Teorem

Hvis A er en symmetrisk positivt definit $n \times n$ matrise så finnes en øvre triangulær $n \times n$ -matrise X slik at:

$$A = X^T X$$

Eksempel

Anvend teoremet over på matrisen

$$A = \begin{bmatrix} 4 & 2 \\ 2 & 3 \end{bmatrix}$$



NTNU

Cholesky- $n \times n$ matrise

Cholesky-faktorisering

Vi har en SPD matrise A , og vil finne en nedre triangulær matrise L slik at:

$$A = LL^T$$

For diagonalelementer:

$$L_{i,i} = \sqrt{A_{i,i} - \sum_{k=1}^{i-1} L_{i,k}^2}$$

For elementer under diagonalen:

$$L_{j,i} = \frac{1}{L_{i,i}} \left(A_{j,i} - \sum_{k=1}^{i-1} L_{j,k} L_{i,k} \right)$$

Vi ser på $n = 4$

$$L = \begin{bmatrix} L_{00} & 0 & 0 & 0 \\ L_{10} & L_{11} & 0 & 0 \\ L_{20} & L_{21} & L_{22} & 0 \\ L_{30} & L_{31} & L_{32} & L_{33} \end{bmatrix}$$

$$LL^T = \begin{bmatrix} L_{00} & 0 & 0 & 0 \\ L_{10} & L_{11} & 0 & 0 \\ L_{20} & L_{21} & L_{22} & 0 \\ L_{30} & L_{31} & L_{32} & L_{33} \end{bmatrix} \begin{bmatrix} L_{00} & L_{10} & L_{20} & L_{30} \\ 0 & L_{11} & L_{21} & L_{31} \\ 0 & 0 & L_{22} & L_{32} \\ 0 & 0 & 0 & L_{33} \end{bmatrix}$$



NTNU

Forts.

$$LL^T = \begin{bmatrix} L_{00}^2 & L_{00}L_{10} & L_{00}L_{20} & L_{00}L_{30} \\ L_{00}L_{10} & L_{10}^2 + L_{11}^2 & L_{10}L_{20} + L_{11}L_{21} & L_{10}L_{30} + L_{11}L_{31} \\ L_{00}L_{20} & L_{10}L_{20} + L_{11}L_{21} & L_{20}^2 + L_{21}^2 + L_{22}^2 & L_{20}L_{30} + L_{21}L_{31} + L_{22}L_{32} \\ L_{00}L_{30} & L_{10}L_{30} + L_{11}L_{31} & L_{20}L_{30} + L_{21}L_{31} + L_{22}L_{32} & L_{30}^2 + L_{31}^2 + L_{32}^2 + L_{33}^2 \end{bmatrix}$$

For diagonalelementer:

$$L_{i,i} = \sqrt{A_{i,i} - \sum_{k=1}^{i-1} L_{i,k}^2}$$

For elementer under diagonalen:

$$L_{j,i} = \frac{1}{L_{i,i}} \left(A_{j,i} - \sum_{k=1}^{i-1} L_{j,k} L_{i,k} \right)$$



NTNU

Cholesky- $n \times n$ matrise, algoritme

Vi har en SPD matrise A :

$$A = \begin{bmatrix} a & \mathbf{b}^T \\ \mathbf{b} & A_1 \end{bmatrix}$$

Vi Cholesky-faktoriserer ved først å finne ut hva som skal stå i første rad/kolonne - Deretter «justerer» vi undermatrisen A_1 og gjør Cholesky-faktorisering på undermatrisen. Vi fortsetter helt til undermatrisen består av kun 1 element

$$X = \begin{bmatrix} \sqrt{a} & \mathbf{u}^T \\ 0 & V \\ 0 & \\ 0 & \end{bmatrix}$$

Her er

$$\mathbf{u} = \frac{\mathbf{b}}{\sqrt{a}}$$

og V er Cholesky-faktoriseringen av:

$$A_1 - \mathbf{u}\mathbf{u}^T$$



NTNU

Cholesky eksempel

finn X slik at $X^T X = A$ for:

$$\begin{bmatrix} 4 & -2 & 2 \\ -2 & 2 & -4 \\ 2 & -4 & 11 \end{bmatrix}$$





NTNU

Cholesky-faktorisering

```
1 import numpy as np
2 def cholesky_decomp(A):
3     (n,n) = A.shape
4     R=np.zeros((n,n))
5     for i in range(n):
6         if A[i,i]<0:
7             raise np.linalg.LinAlgError(
8                 "A er ikke positiv definit"
9             )
10        R[i,i] = np.sqrt(A[i,i])
11        u = np.array(A[i,i+1:])/R[i,i]
12        R[i,i+1:n] = u
13        A[i+1:n,i+1:n] = A[i+1:n,i+1:n]-u.T@ u
14    return R
```

Hvorfor?

- Er det noe kjent med faktoriseringen $A = X^T X$ når X er en øvre triangulær matrise?
- Det stemmer! - Det er en type *LU*-faktorisering, og vi kan bruke den sådan :)
- En bonus er at X er raskere å regne ut og bruke enn L og U (ca. dobbelt så rask)



NTNU

SPD matriser og Indreprodukt

Indreprodukt: Definisjon

La V være et vektorrom over $\mathbb{R}$ (reelle skalarer). Et indreprodukt (u, v) er da en funksjon som tar to vektorer i V og gir tilbake reelt tall ($V \times V \rightarrow \mathbb{R}$), og har følgende egenskaper

1. $\forall u \in V, (u, u) \geq 0$ og $(u, u) = 0 \Leftrightarrow u = 0$
2. $\forall u, v \in V, (u, v) = (v, u)$
3. $\forall u, v, w \in V$ og for alle skalarer $a, b \in \mathbb{R}$ er $(au + bv, w) = a(u, w) + b(v, w)$

Nytt indreprodukt

For en symmetrisk positivt definit matrise A , vil funksjonen

$$(u, v)_A = u^T A v$$

tilfredsstille kravende for et indreprodukt! Vi kan således bruke det til å beskrive lengder av vektorer (normer) og retninger (feks ortogonalitet)!



NTNU

SPD matriser og Indreprodukt

Vi undersøker

Indreprodukt: Definisjon

La V være et vektorrom over $\mathbb{R}$ (reelle skalarer). Et indreprodukt (u, v) er da en funksjon som tar to vektorer i V og gir tilbake reelt tall ($V \times V \rightarrow \mathbb{R}$), og har følgende egenskaper

1. $\forall u \in V, (u, u) \geq 0$ og $(u, u) = 0 \Leftrightarrow u = 0$
2. $\forall u, v \in V, (u, v) = (v, u)$
3. $\forall u, v, w \in V$ og for alle skalarer $a, b \in \mathbb{R}$ er $(au + bv, w) = a(u, w) + b(v, w)$

$$(u, v)_A = u^T A v$$



NTNU

Ortonormt basis med $(,)_A$

Vi har altså en symmetrisk positivt definit matrise A sammen med et spesialindreprodukt $(u, v)_A = u^T A v$.

Kan vi bygge et ortonormt basis med denne?

Ja, og vi kan bruke Gram-Schmidt på samme måte, bare med det nye indreproduktet

Vi sier da gjerne at vektorene i basis er *konjugerte* eller *A-ortogonale*

$$B = \{p_1, p_2, \dots, p_n\}$$

$$(p_i, p_i)_A = p_i^T A p_i = 1$$

$$(p_i, p_j)_A = p_i^T A p_j = 0$$



NTNU

Konjugerte gradienter

Setter vi dette inn i ligningen vår får vi:

- Anta at vi har et $n \times n$ likningssystem $\mathbf{Ax} = \mathbf{b}$ hvor $\mathbf{A}$ er SPD.
- Vi antar også at vi har et sett $P = \{\mathbf{p}_1, \mathbf{p}_2, \dots, \mathbf{p}_n\}$ med n parvis konjugerte/A-ortogonale vektorer.
- Vektorene i P danner da en basis for $\mathbb{R}^n$.

Dersom vi vil uttrykke løsningen vår x_s i denne basisen blir det:



NTNU

Konjugerte gradienter

Setter vi dette inn i ligningen vår får vi:

- Anta at vi har et $n \times n$ likningssystem $\mathbf{Ax} = \mathbf{b}$ hvor $\mathbf{A}$ er SPD.
- Vi antar også at vi har et sett $P = \{\mathbf{p}_1, \mathbf{p}_2, \dots, \mathbf{p}_n\}$ med n parvis konjugerte/A-ortogonale vektorer.
- Vektorene i P danner da en basis for $\mathbb{R}^n$.

Dersom vi vil uttrykke løsningen vår x_s i denne basisen blir det:

$$\mathbf{x}_s = \sum_{i=1}^n \alpha_i \mathbf{p}_i$$



NTNU

Konjugerte gradienter

- Anta at vi har et $n \times n$ likningssystem $\mathbf{Ax} = \mathbf{b}$ hvor $\mathbf{A}$ er SPD.
- Vi antar også at vi har et sett $P = \{\mathbf{p}_1, \mathbf{p}_2, \dots, \mathbf{p}_n\}$ med n parvis konjugerte/A-ortogonale vektorer.
- Vektorene i P danner da en basis for $\mathbb{R}^n$.

Dersom vi vil uttrykke løsningen vår x_s i denne basisen blir det:

$$\mathbf{x}_s = \sum_{i=1}^n \alpha_i \mathbf{p}_i$$

Setter vi dette inn i ligningen vår får vi:

$$\begin{aligned} \mathbf{Ax}_s &= \mathbf{b} \\ \mathbf{A} \sum_{i=1}^n \alpha_i \mathbf{p}_i &= \mathbf{b} \end{aligned}$$



NTNU

Konjugerte gradienter

- Anta at vi har et $n \times n$ likningssystem $\mathbf{Ax} = \mathbf{b}$ hvor $\mathbf{A}$ er SPD.
- Vi antar også at vi har et sett $P = \{\mathbf{p}_1, \mathbf{p}_2, \dots, \mathbf{p}_n\}$ med n parvis konjugerte/A-ortogonale vektorer.
- Vektorene i P danner da en basis for $\mathbb{R}^n$.

Dersom vi vil uttrykke løsningen vår x_s i denne basisen blir det:

$$\mathbf{x}_s = \sum_{i=1}^n \alpha_i \mathbf{p}_i$$

Setter vi dette inn i ligningen vår får vi:

$$\mathbf{Ax}_s = \mathbf{b}$$

$$\mathbf{A} \sum_{i=1}^n \alpha_i \mathbf{p}_i = \mathbf{b}$$

$$\sum_{i=1}^n \alpha_i \mathbf{A} \mathbf{p}_i = \mathbf{b}$$



NTNU

Konjugerte gradienter

- Anta at vi har et $n \times n$ likningssystem $\mathbf{Ax} = \mathbf{b}$ hvor $\mathbf{A}$ er SPD.
- Vi antar også at vi har et sett $P = \{\mathbf{p}_1, \mathbf{p}_2, \dots, \mathbf{p}_n\}$ med n parvis konjugerte/A-ortogonale vektorer.
- Vektorene i P danner da en basis for $\mathbb{R}^n$.

Dersom vi vil uttrykke løsningen vår x_s i denne basisen blir det:

$$\mathbf{x}_s = \sum_{i=1}^n \alpha_i \mathbf{p}_i$$

Setter vi dette inn i ligningen vår får vi:

$$\mathbf{Ax}_s = \mathbf{b}$$

$$\mathbf{A} \sum_{i=1}^n \alpha_i \mathbf{p}_i = \mathbf{b}$$

$$\sum_{i=1}^n \alpha_i \mathbf{A} \mathbf{p}_i = \mathbf{b}$$

$$\sum_{i=1}^n \alpha_i \mathbf{p}_k^T \mathbf{A} \mathbf{p}_i = \mathbf{p}_k^T \mathbf{b}$$



NTNU

Konjugerte gradienter

- Anta at vi har et $n \times n$ likningssystem $\mathbf{Ax} = \mathbf{b}$ hvor $\mathbf{A}$ er SPD.
- Vi antar også at vi har et sett $P = \{\mathbf{p}_1, \mathbf{p}_2, \dots, \mathbf{p}_n\}$ med n parvis konjugerte/A-ortogonale vektorer.
- Vektorene i P danner da en basis for $\mathbb{R}^n$.

Dersom vi vil uttrykke løsningen vår x_s i denne basisen blir det:

$$\mathbf{x}_s = \sum_{i=1}^n \alpha_i \mathbf{p}_i$$

Setter vi dette inn i ligningen vår får vi:

$$\mathbf{Ax}_s = \mathbf{b}$$

$$\mathbf{A} \sum_{i=1}^n \alpha_i \mathbf{p}_i = \mathbf{b}$$

$$\sum_{i=1}^n \alpha_i \mathbf{A} \mathbf{p}_i = \mathbf{b}$$

$$\sum_{i=1}^n \alpha_i \mathbf{p}_k^T \mathbf{A} \mathbf{p}_i = \mathbf{p}_k^T \mathbf{b}$$

$$\sum_{i=1}^n \alpha_i (\mathbf{p}_k^T, \mathbf{p}_i)_A = \mathbf{p}_k^T \mathbf{b}$$



NTNU

Konjugerte gradienter

- Anta at vi har et $n \times n$ likningssystem $\mathbf{Ax} = \mathbf{b}$ hvor $\mathbf{A}$ er SPD.
- Vi antar også at vi har et sett $P = \{\mathbf{p}_1, \mathbf{p}_2, \dots, \mathbf{p}_n\}$ med n parvis konjugerte/A-ortogonale vektorer.
- Vektorene i P danner da en basis for $\mathbb{R}^n$.

Dersom vi vil uttrykke løsningen vår x_s i denne basisen blir det:

$$\mathbf{x}_s = \sum_{i=1}^n \alpha_i \mathbf{p}_i$$

Setter vi dette inn i ligningen vår får vi:

$$\mathbf{Ax}_s = \mathbf{b}$$

$$\mathbf{A} \sum_{i=1}^n \alpha_i \mathbf{p}_i = \mathbf{b}$$

$$\sum_{i=1}^n \alpha_i \mathbf{A} \mathbf{p}_i = \mathbf{b}$$

$$\sum_{i=1}^n \alpha_i \mathbf{p}_k^T \mathbf{A} \mathbf{p}_i = \mathbf{p}_k^T \mathbf{b}$$

$$\sum_{i=1}^n \alpha_i (\mathbf{p}_k^T, \mathbf{p}_i)_A = \mathbf{p}_k^T \mathbf{b}$$

$$\alpha_k (\mathbf{p}_k, \mathbf{p}_k)_A = \mathbf{p}_k^T \mathbf{b}$$

Konjugerte gradienter

- Anta at vi har et $n \times n$ likningssystem $\mathbf{Ax} = \mathbf{b}$ hvor $\mathbf{A}$ er SPD.
- Vi antar også at vi har et sett $P = \{\mathbf{p}_1, \mathbf{p}_2, \dots, \mathbf{p}_n\}$ med n parvis konjugerte/A-ortogonale vektorer.
- Vektorene i P danner da en basis for $\mathbb{R}^n$.

Dersom vi vil uttrykke løsningen vår $\mathbf{x}_s$ i denne basisen blir det:

$$\mathbf{x}_s = \sum_{i=1}^n \alpha_i \mathbf{p}_i$$

Setter vi dette inn i ligningen vår får vi:

$$\mathbf{Ax}_s = \mathbf{b}$$

$$\mathbf{A} \sum_{i=1}^n \alpha_i \mathbf{p}_i = \mathbf{b}$$

$$\sum_{i=1}^n \alpha_i \mathbf{A} \mathbf{p}_i = \mathbf{b}$$

$$\sum_{i=1}^n \alpha_i \mathbf{p}_k^T \mathbf{A} \mathbf{p}_i = \mathbf{p}_k^T \mathbf{b}$$

$$\sum_{i=1}^n \alpha_i (\mathbf{p}_k^T, \mathbf{p}_i)_A = \mathbf{p}_k^T \mathbf{b}$$

$$\alpha_k (\mathbf{p}_k, \mathbf{p}_k)_A = \mathbf{p}_k^T \mathbf{b}$$

$$\alpha_k = \frac{\mathbf{p}_k^T \mathbf{b}}{(\mathbf{p}_k, \mathbf{p}_k)_A} = \frac{(\mathbf{p}_k, \mathbf{b})}{(\mathbf{p}_k', \mathbf{p}_k)_A}$$

Konjugerte gradienter

Strategi

For å løse likningssystemet kan vi altså se etter et sett med n konjugerte retninger $\mathbf{p}_i$, og regne ut koefisientene i linjærkombinasjonen

$$\mathbf{x}_s = \sum_{i=1}^n \alpha_i \mathbf{p}_i$$

via

$$\alpha_k = \frac{(\mathbf{p}_k, \mathbf{b})}{(\mathbf{p}_k, \mathbf{p}_k)_A} = \frac{(\mathbf{p}_k, \mathbf{b})}{(\mathbf{p}_k, \mathbf{A}\mathbf{p}_k)} = \frac{\mathbf{p}_k^T \mathbf{b}}{\mathbf{p}_k^T \mathbf{A} \mathbf{p}_k}$$

for $k = 1, 2, 3, \dots, n$



NTNU

Konjugerte gradienter

Strategi

For å løse likningssystemet kan vi altså se etter et sett med n konjugerte retninger $\mathbf{p}_i$, og regne ut koefisientene i linjærkombinasjonen

$$\mathbf{x}_s = \sum_{i=1}^n \alpha_i \mathbf{p}_i$$

via

$$\alpha_k = \frac{(\mathbf{p}_k, \mathbf{b})}{(\mathbf{p}_k, \mathbf{p}_k)_A} = \frac{(\mathbf{p}_k, \mathbf{b})}{(\mathbf{p}_k, \mathbf{A}\mathbf{p}_k)} = \frac{\mathbf{p}_k^T \mathbf{b}}{\mathbf{p}_k^T \mathbf{A}\mathbf{p}_k}$$

for $k = 1, 2, 3, \dots, n$

Hvor finner vi de konjugerte retningene?

Et bra sted å lete etter de konjugerte vektorene er i *Krylov underrommet*:

$$\mathcal{K} = \{\mathbf{b}, \mathbf{A}\mathbf{b}, \mathbf{A}^2\mathbf{b}, \dots, \mathbf{A}^r\mathbf{b}\}$$



NTNU

Konjugerte gradienter

Strategi

For å løse likningssystemet kan vi altså se etter et sett med n konjugerte retninger $\mathbf{p}_i$, og regne ut koefisientene i linjærkombinasjonen

$$\mathbf{x}_s = \sum_{i=1}^n \alpha_i \mathbf{p}_i$$

via

$$\alpha_k = \frac{(\mathbf{p}_k, \mathbf{b})}{(\mathbf{p}_k, \mathbf{p}_k)_A} = \frac{(\mathbf{p}_k, \mathbf{b})}{(\mathbf{p}_k, \mathbf{A}\mathbf{p}_k)} = \frac{\mathbf{p}_k^T \mathbf{b}}{\mathbf{p}_k^T \mathbf{A}\mathbf{p}_k}$$

for $k = 1, 2, 3, \dots, n$

Hvor finner vi de konjugerte retningene?

Et bra sted å lete etter de konjugerte vektorene er i *Krylov underrommet*:

$$\mathcal{K} = \{\mathbf{b}, \mathbf{A}\mathbf{b}, \mathbf{A}^2\mathbf{b}, \dots, \mathbf{A}^r\mathbf{b}\}$$

Konjugerte gradienter

- Strategien vår er en type *direkte metode*
- Dersom vi er lur i valgene vår av $\mathbf{p}_k$ trenger vi ikke å finne alle n retningene
- Vi kan også formulere det som en *iterativ metode*



NTNU

Først noen teoremer

Matrisekalkulus

Vi har en SPD matrise $\mathbf{A}$ og en skalar funksjon:

$$f(\mathbf{x}) = \frac{1}{2} \mathbf{x}^T \mathbf{A} \mathbf{x} - \mathbf{x}^T \mathbf{b}$$



NTNU

Først noen teoremer

Matrisekalkulus

Vi har en SPD matrise $\mathbf{A}$ og en skalar funksjon:

$$f(\mathbf{x}) = \frac{1}{2} \mathbf{x}^T \mathbf{A} \mathbf{x} - \mathbf{x}^T \mathbf{b}$$

det kan vises at:

$$\frac{\partial f(\mathbf{x})}{\partial \mathbf{x}} = \frac{1}{2} (\mathbf{A} + \mathbf{A}^T) \mathbf{x} - \mathbf{b}$$



NTNU

Først noen teoremer

Matrisekalkulus

Vi har en SPD matrise $\mathbf{A}$ og en skalar funksjon:

$$f(\mathbf{x}) = \frac{1}{2} \mathbf{x}^T \mathbf{A} \mathbf{x} - \mathbf{x}^T \mathbf{b}$$

det kan vises at:

$$\frac{\partial f(\mathbf{x})}{\partial \mathbf{x}} = \frac{1}{2} (\mathbf{A} + \mathbf{A}^T) \mathbf{x} - \mathbf{b}$$

Og dersom $\mathbf{A}$ er SPD:

$$\frac{\partial f(\mathbf{x})}{\partial \mathbf{x}} = 2 \cdot \frac{1}{2} (\mathbf{A}) \mathbf{x} - \mathbf{b} = \mathbf{A} \mathbf{x} - \mathbf{b}$$

$\frac{\partial f(\mathbf{x})}{\partial \mathbf{x}}$ kalles *gradienten* til f , $\nabla f(\mathbf{x})$



NTNU

Først noen teoremer

Matrisekalkulus

Vi har en SPD matrise $\mathbf{A}$ og en skalar funksjon:

$$f(\mathbf{x}) = \frac{1}{2} \mathbf{x}^T \mathbf{A} \mathbf{x} - \mathbf{x}^T \mathbf{b}$$

det kan vises at:

$$\frac{\partial f(\mathbf{x})}{\partial \mathbf{x}} = \frac{1}{2} (\mathbf{A} + \mathbf{A}^T) \mathbf{x} - \mathbf{b}$$

Og dersom $\mathbf{A}$ er SPD:

$$\frac{\partial f(\mathbf{x})}{\partial \mathbf{x}} = 2 \cdot \frac{1}{2} (\mathbf{A}) \mathbf{x} - \mathbf{b} = \mathbf{A} \mathbf{x} - \mathbf{b}$$

$\frac{\partial f(\mathbf{x})}{\partial \mathbf{x}}$ kalles *gradienten* til f , $\nabla f(\mathbf{x})$

$$\nabla f(\mathbf{x}) = \left[\frac{\partial f}{\partial x_1}, \frac{\partial f}{\partial x_2}, \dots, \frac{\partial f}{\partial x_n} \right]^T$$

Gradienten forteller oss i hvilket retning f stiger raskest, og hvor raskt den stiger. Vi vil egentlig løse $\mathbf{A} \mathbf{x} = \mathbf{b} \Leftrightarrow \mathbf{A} \mathbf{x} - \mathbf{b} = 0$, altså $\nabla f(\mathbf{x}) = 0$

Vi kan altså løse det *duale* problemet: Å finne ekstremalpunktet til den kvadratiske flervariabel funksjonen $f(x_1, x_2, \dots, x_n)$. Og når $\mathbf{A}$ er SPD:



NTNU

Først noen teoremer

Matrisekalkulus

Vi har en SPD matrise $\mathbf{A}$ og en skalar funksjon:

$$f(\mathbf{x}) = \frac{1}{2} \mathbf{x}^T \mathbf{A} \mathbf{x} - \mathbf{x}^T \mathbf{b}$$

det kan vises at:

$$\frac{\partial f(\mathbf{x})}{\partial \mathbf{x}} = \frac{1}{2} (\mathbf{A} + \mathbf{A}^T) \mathbf{x} - \mathbf{b}$$

Og dersom $\mathbf{A}$ er SPD:

$$\frac{\partial f(\mathbf{x})}{\partial \mathbf{x}} = 2 \cdot \frac{1}{2} (\mathbf{A}) \mathbf{x} - \mathbf{b} = \mathbf{A} \mathbf{x} - \mathbf{b}$$

$\frac{\partial f(\mathbf{x})}{\partial \mathbf{x}}$ kalles *gradienten* til f , $\nabla f(\mathbf{x})$

$$\nabla f(\mathbf{x}) = \left[\frac{\partial f}{\partial x_1}, \frac{\partial f}{\partial x_2}, \dots, \frac{\partial f}{\partial x_n} \right]^T$$

Gradienten forteller oss i hvilket retning f stiger raskest, og hvor raskt den stiger. Vi vil egentlig løse $\mathbf{A} \mathbf{x} = \mathbf{b} \Leftrightarrow \mathbf{A} \mathbf{x} - \mathbf{b} = 0$, altså $\nabla f(\mathbf{x}) = 0$

Vi kan altså løse det *duale* problemet: Å finne ekstremalpunktet til den kvadratiske flervariabel funksjonen $f(x_1, x_2, \dots, x_n)$. Og når $\mathbf{A}$ er SPD:

- har f ett unikt ekstremalpunkt



NTNU

Først noen teoremer

Matrisekalkulus

Vi har en SPD matrise $\mathbf{A}$ og en skalar funksjon:

$$f(\mathbf{x}) = \frac{1}{2} \mathbf{x}^T \mathbf{A} \mathbf{x} - \mathbf{x}^T \mathbf{b}$$

det kan vises at:

$$\frac{\partial f(\mathbf{x})}{\partial \mathbf{x}} = \frac{1}{2} (\mathbf{A} + \mathbf{A}^T) \mathbf{x} - \mathbf{b}$$

Og dersom $\mathbf{A}$ er SPD:

$$\frac{\partial f(\mathbf{x})}{\partial \mathbf{x}} = 2 \cdot \frac{1}{2} (\mathbf{A}) \mathbf{x} - \mathbf{b} = \mathbf{A} \mathbf{x} - \mathbf{b}$$

$\frac{\partial f(\mathbf{x})}{\partial \mathbf{x}}$ kalles *gradienten* til f , $\nabla f(\mathbf{x})$

$$\nabla f(\mathbf{x}) = \left[\frac{\partial f}{\partial x_1}, \frac{\partial f}{\partial x_2}, \dots, \frac{\partial f}{\partial x_n} \right]^T$$

Gradienten forteller oss i hvilket retning f stiger raskest, og hvor raskt den stiger. Vi vil egentlig løse $\mathbf{A} \mathbf{x} = \mathbf{b} \Leftrightarrow \mathbf{A} \mathbf{x} - \mathbf{b} = 0$, altså $\nabla f(\mathbf{x}) = 0$

Vi kan altså løse det *duale* problemet: Å finne ekstremalpunktet til den kvadratiske flervariabel funksjonen $f(x_1, x_2, \dots, x_n)$. Og når $\mathbf{A}$ er SPD:

- har f ett unikt ekstremalpunkt
- og det er et bunnpunkt

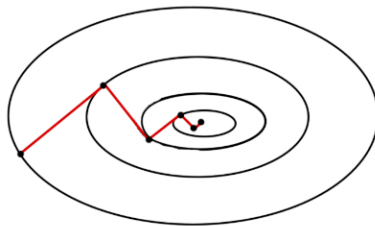


NTNU

Steepest Gradient Descent

Vi vil løse $\mathbf{Ax} = \mathbf{b}$ ved å finne nullpunktet til $f(\mathbf{x}) = \frac{1}{2}\mathbf{x}^T \mathbf{Ax} - \mathbf{x}^T \mathbf{b}$. Residualen $\mathbf{r}_k = \mathbf{Ax}_k - \mathbf{b}$ peker hele tiden i retningen hvor f stiger raskest - vi vil lete i motsatt retning

- Vi begynner i $\mathbf{x}_0$, og beveger oss i retning $-\nabla f(\mathbf{x}_0) = -(\mathbf{Ax} - \mathbf{b}) = \mathbf{r}_0$
- Vi søker langs $-\mathbf{r}_0$ helt til $\mathbf{x}_1$ hvor vi ikke lenger flytter oss nedover: $\mathbf{r}_1 \perp \mathbf{r}_0$
- Ny søkeretning er da $\mathbf{r}_1$ - rinse and repeat





NTNU

Konjugerte gradienter

- Vi begynner i $\mathbf{x}_0$ og beveger oss i retningen til residualen: $\mathbf{p}_0 = \mathbf{b} - \mathbf{A}\mathbf{x}_0$
- I steg k blir det nye punktet $\mathbf{x}_{k+1} = \mathbf{x}_k + \alpha_k \mathbf{p}_k$. Her gir α_k hvor langt langs $\mathbf{p}_k$ vi skal gå

For å beregne α_k minimerer vi $f(\mathbf{x}_{k+1})$, vi løser:

$$\frac{\partial f(\mathbf{x}_{k+1})}{\partial \alpha_k} = 0$$

for α_k og får:

$$\alpha_k = \frac{\mathbf{p}_k^T (\mathbf{b} - \mathbf{A}\mathbf{x}_k)}{\mathbf{p}_k^T \mathbf{A} \mathbf{p}_k} = \frac{(\mathbf{p}_k, \mathbf{r}_k)}{(\mathbf{p}_k, \mathbf{A} \mathbf{p}_k)}$$

- I gradient descent blir nå neste retning vi søker langs $\mathbf{r}_{k+1}$. Men i konjugerte gradienters metode krever vi at neste retning $\mathbf{p}_{k+1}$ også skal være A-konjugert med alle de forrige søkeretningene!
- Vi fjerner altså alt av $\mathbf{r}_{k+1}$ som peker i samme retning som de forrige retningene vi har gått i = Gram-Schmidt med det nye indreproduktet vårt!

$$\mathbf{p}_{k+1} = \mathbf{r}_{k+1} - \sum_{i \leq k} \frac{(\mathbf{p}_i, \mathbf{r}_{k+1})_A}{(\mathbf{p}_i, \mathbf{p}_i)_A} \mathbf{p}_i$$

- Fortsett til $\|\mathbf{r}_k\|$ er liten nok



NTNU

Konjugerte gradienters metode - python

```
import numpy as np
def conjugate_gradient(A, b, x0, tol, N):
    rk = b-A @ x0
    pk = rk
    err = np.linalg.norm(rk)
    it = 0
    xold = x0
    while (err > tol and it < N):
        alpha = (rk.T @ rk)/(pk.T @ A @ pk)
        xnew = xold + alpha*pk
        rk2 = rk-alpha*A @ pk
        beta = (rk2.T @ rk2)/(rk.T @ rk)
        pk = rk+beta*pk
        xold = xnew
        rk = rk2
        err = np.linalg.norm(rk)
        it+=1

    return xnew, err, it
```

- I praksis bruker vi *ikke* Gram-Schmidt
- Da måtte vi ha lagret alle tidligere konjugerte retninger, det kan vi trikse oss vekk fra
- $p_{k+1} = r_{k+1} + \beta_k p_k$
- $\beta_k = \frac{r_{k+1}^T r_{k+1}}{r_k^T r_k}$



NTNU

CG - konvergens

- ▶ Konjugerte gradienters metode er *egentlig* en direkte metode
- ▶ Etter n iterasjoner skulle vi gå tom for konjugerte retninger slik at residualen blir 0
- ▶ I praksis kan A være dårlig kondisjonert, slik at avrundingsfeil gjør at vi bruker mange flere iterasjoner
- ▶ Det kan også være årsak til at CG blir ustabil
- ▶ En løsning her kan være å *førkondisjonere* A



NTNU

Førkondisjonering

- ▶ Ved førkondisjonering prøver vi å finne en matrise M som:
 1. Ligner mest mulig på A
 2. Er lett å invertere (finne M^{-1})
- ▶ Vi forsøker da heller å løse systemet $M^{-1}Ax = M^{-1}b$
- ▶ $M^{-1}A$ skulle være mer lik identitetsmatrisen, ha lavere kondisjonstall og CG-konvergerer raskere
- ▶ For CG med SPD-matriser er *delvis cholesky-faktorisering* et vanlig valg for førkondisjonering